

24-HOUR HACKATHON

BLACK BOX HACKATHON

PROBLEM STATEMENTS

PROBLEM STATEMENT 1

STM32-BASED INDUSTRIAL IoT DATA ACQUISITION DEVICE

24-Hour Hackathon

Background

Industrial machines use sensors to measure parameters such as temperature, pressure, flow, level, and vibration.

Many of these sensors provide standard outputs such as 4–20 mA and 0–5 V. These signals need to be properly conditioned and protected before they can be read by a microcontroller.

Your task is to design an STM32-based IoT device that can read these sensor signals and send the measured data wirelessly.

In simple words: Design the schematic and PCB for a device that reads industrial sensor signals using STM32 and sends the data over Wi-Fi.

The Challenge

Design an STM32 F-Series based IoT device with:

- At least one 4–20 mA input
- At least one 0–5 V input
- Suitable signal conditioning and protection
- A stable and protected power supply
- Wi-Fi connectivity
- An STM32 section capable of data acquisition and processing

The design should show how an input signal is received, conditioned for the STM32, and routed to the Wi-Fi interface for transmission.

What You Need to Design

1 • STM32 Controller

Use an STM32 F-Series microcontroller as the main controller. The schematic should show its supply, decoupling, clock, reset, programming/debug header, and the pins used for the sensor inputs and the Wi-Fi interface.

2 • 4–20 mA Input

Your design must include an interface for a 4–20 mA industrial sensor. The circuit should convert and condition the current signal so that it can be safely measured by the STM32.

3 • 0–5 V Input

Your design must also include an interface for a 0–5 V analog sensor. The input should be safely scaled and conditioned for the STM32 ADC.

4 • Power Supply

Design a stable and protected power supply. Consider voltage regulation, reverse-polarity protection, over-current/short-circuit protection, over-voltage protection, and filtering/decoupling.

5 • Wi-Fi Section

The design must include a Wi-Fi interface for transmitting the acquired sensor data, with its supply, the connection to the STM32.

6 • PCB Layout

Produce a complete PCB layout for the circuit. Consider board outline and connector placement, layer stack-up, track widths, ground and power planes, separation of analog and digital sections, and component placement.

Example

Consider a pressure sensor that provides a 4–20 mA output. Your schematic should show how that signal is converted and conditioned into a voltage the STM32 ADC can measure, how the STM32 connects to the Wi-Fi interface, and how the whole circuit is powered and protected. The same board should also have an input for a 0–5 V sensor.

Expected Demonstration

- Complete circuit schematic
- STM32 F-Series controller section
- 4–20 mA input circuit
- 0–5 V input circuit
- Signal conditioning and protection
- Power supply section with protection
- Wi-Fi interface section
- Complete PCB layout with routing
- Bill of materials

Teams present and explain their design files.

Expected Deliverables

Schematic

- Complete circuit schematic
- Component values and part numbers
- Clear labelling of inputs, power, and interfaces

PCB

- Complete PCB layout
- Routing and layer stack-up
- Component placement
- Board outline and connector placement

Documentation

- Bill of materials (BOM)
- Brief explanation of the design choices

Evaluation Criteria

Category	What We Will Look At
Schematic Quality	Correctness, completeness and readability
STM32 Section	MCU selection and supporting circuit design
4–20 mA Input	Correct conversion and conditioning circuit
0–5 V Input	Correct scaling and conditioning circuit
Signal Conditioning	Proper conversion and protection
Power Supply	Stability and protection
Wi-Fi Section	Correct interface and antenna considerations
PCB Design	Layout, routing, planes and placement
Component Selection	Suitability of parts and quality of the BOM

What a Good Solution Looks Like

Design a reliable STM32-based board that can read 4–20 mA and 0–5 V sensor signals, safely condition them, and send the measured data over Wi-Fi — delivered as a complete schematic, PCB layout and BOM.

Focus on making the core design correct and manufacturable rather than adding unnecessary features.

BEST OF LUCK!

PROBLEM STATEMENT 2

LOCATION-AWARE REMOTE CONTENT MANAGEMENT

24-Hour Hackathon

Background

Imagine a company has hundreds of digital display devices installed at different locations. Updating each device manually would take a lot of time.

The company needs a central platform where an administrator can manage devices, upload content, and decide what each device should display without physically visiting it.

The challenge becomes more interesting when the content depends on where the device is located.

The right content should reach the right device at the right location.

The Challenge

Build a centralized platform that allows an authorized administrator to remotely manage multiple display devices and their content.

- Add and manage display devices
- Configure device settings remotely
- Upload, modify, delete, and schedule content
- Manage images, videos, and other digital content
- Create geographical regions or zones
- Assign content to specific regions
- Monitor device status and connectivity

The display device should automatically determine its location and show the content assigned to that location.

What You Need to Build

1 • Device Management

The administrator should be able to manage multiple display devices and view basic information such as device ID, status, connectivity, location, and assigned content. Unauthorized users should not be able to access device controls.

2 • Content Management

Administrators should be able to upload, replace, delete, schedule, and assign images and videos to devices or locations.

3 • Location-Based Content

Administrators should be able to create geographical zones and assign content to them. When a display device enters a zone, it should automatically show the content configured for that location. If it moves to another zone, the content should update.

4 • Remote Monitoring

The administrator should be able to see whether devices are online or offline, connected, reporting their location, and receiving the latest content. The system should handle temporary network interruptions gracefully.

5 • Security & Access Control

Only authorized administrators should be able to manage devices, content, and settings.

Example

A company has display devices on vehicles travelling between cities. Chandigarh can have one set of promotional content, Delhi another, and Mumbai another. When the device moves from Chandigarh to Delhi, the displayed content should change automatically.

Expected Demonstration

- Multiple display devices connected to a central platform
- Remote device management
- Uploading and managing content
- Creating geographical zones
- Assigning different content to different zones
- Automatic content selection based on device location
- Device status monitoring
- Basic access control

Devices can be physical displays, simulated devices, or a combination of both.

Expected Deliverables

Platform

- Working web or desktop management interface
- Device management
- Content management
- Location/zone management

Device

- Display client/device application
- Location detection
- Content retrieval and display

Documentation

- System architecture
- Basic setup instructions
- Location-based content logic

Evaluation Criteria

Category	What We Will Look At
Device Management	Ability to manage multiple devices
Content Management	Upload, modify, delete and schedule content
Location Awareness	Correct location-based content selection
Remote Control	Central changes reach the devices
Reliability	Behaviour during network interruptions
Security	Access control and protection
User Experience	Ease of managing devices and content
Scalability	Ability to support more devices and content

What a Good Solution Looks Like

Build a platform where an administrator can remotely manage multiple display devices and automatically control what they show based on their geographical location.

Focus on reliable device management, content management, and location-based switching.

BEST OF LUCK!

PROBLEM STATEMENT 3

AI AGENT FOR AUTONOMOUS EMBEDDED FIRMWARE TESTING

24-Hour Hackathon

Background

Testing embedded firmware usually requires engineers to manually write test cases, set up hardware, run the firmware, check the output, and investigate failures.

Firmware can behave differently when a sensor is disconnected, a value goes outside its expected range, communication is interrupted, or the system enters an unexpected state.

The goal is to make firmware testing smarter, faster, and less dependent on manually written test cases.

The Challenge

Build an AI agent that can understand embedded firmware and automatically test it using a virtual hardware environment.

- Read and understand embedded firmware
- Identify important conditions to test
- Generate meaningful test scenarios
- Run tests in a hardware simulator
- Observe firmware output
- Identify unexpected behaviour or failures
- Generate a report explaining what was tested and what went wrong

The agent should not only generate test code. It should decide what is worth testing and evaluate the firmware's behaviour.

What You Need to Build

1 • Firmware Understanding

The agent should analyse the firmware and identify inputs/outputs, sensors and peripherals, important conditions, error handling, state changes, and communication interfaces.

2 • Automatic Test Generation

Generate tests for normal conditions, boundary values, abnormal sensor values, sensor disconnection, communication failure, unexpected inputs, state transitions, and combinations of conditions.

3 • Virtual Hardware Testing

Execute the generated tests using a simulator such as Wokwi, Renode, or another suitable environment. The agent should be able to provide inputs such as sensor values, button inputs, communication events, or hardware failures.

4 • Observe & Evaluate

Observe outputs such as UART/Serial logs, GPIO states, sensor responses, peripheral outputs, error messages, or system state, then decide whether the behaviour matches what was expected.

5 • Automated Test Report

The report should include tests generated and executed, passed/failed tests, expected and observed behaviour, possible causes, and relevant firmware sections where possible.

Example

Consider firmware that controls a cooling fan based on temperature: fan OFF below 30°C, ON above 30°C, and an error if the sensor stops responding. The agent could test 25°C, 30°C, 31°C, rapid temperature changes, sensor disconnection, and invalid readings, then report any incorrect behaviour.

Expected Demonstration

- Provide embedded firmware to the system
- AI analyses the firmware
- AI generates test scenarios
- Tests run in a virtual hardware environment
- Firmware behaviour is observed
- Failures are identified
- A test/debugging report is generated

Expected Deliverables

AI Agent

- Firmware-analysis system
- Automatic test generation
- Test execution workflow
- Failure detection and analysis

Simulation

- Integration with Wokwi, Renode, or another simulator
- At least one meaningful embedded firmware example

Report

- Generated test cases
- Test results
- Detected failures
- Explanation of identified issues

Evaluation Criteria

Category	What We Will Look At
Firmware Understanding	How well the agent understands the code
Test Generation	Relevance and coverage of tests
Edge Cases	Ability to find abnormal conditions
Simulation	Successful automated test execution
Failure Detection	Ability to identify incorrect behaviour
Analysis	Quality of failure explanation
Automation	How little manual intervention is required
User Experience	Ease of using the testing system

What a Good Solution Looks Like

Build an AI agent that can read embedded firmware, decide what should be tested, automatically run those tests in a virtual hardware environment, and identify when the firmware behaves incorrectly.

Focus on a working autonomous testing loop rather than simply building an AI chatbot that explains code.

BEST OF LUCK!

PROBLEM STATEMENT 4

PREDICTIVE MAINTENANCE FOR INDUSTRIAL MOTORS

24-Hour Hackathon

Background

Industrial motors are often maintained only after they fail or according to a fixed maintenance schedule.

Unexpected failures can cause downtime and production loss, while replacing healthy components wastes time and money.

Before a motor fails, it often shows early signs such as increased vibration, unusual sound, higher current, or rising temperature.

The challenge is to detect these changes early and warn the operator before a major failure occurs.

Can you detect a motor problem before the motor actually fails?

The Challenge

Build a low-cost, sensor-based predictive maintenance system for an industrial motor.

- Vibration
- Sound
- Current
- Temperature

Your system should monitor the motor during operation, learn what normal operation looks like, and detect when its behaviour changes because of an introduced fault.

You may use one or more sensing methods such as:

What You Need to Build

1 • Motor Monitoring

Continuously collect data from at least one sensor while the motor is running. The system should first establish a normal operating baseline.

2 • Fault Detection

Introduce a controlled fault or change in operating condition and detect the resulting change in sensor data. Possible test conditions include:

- Adding imbalance
- Increasing the load
- Blocking airflow to cause heating
- Another safe, controlled abnormal condition

The system should distinguish between normal and abnormal behaviour.

3 • Health Status

Clearly show the current motor condition, for example NORMAL, WARNING, or CRITICAL. The alert can use an LED, buzzer, display, or dashboard.

Example

Suppose a motor normally operates with a certain vibration level. After adding a small controlled imbalance, the vibration increases. Your system should detect the change from the normal baseline and generate a warning.

The main demonstration should use a live, physically introduced fault rather than only pre-recorded data.

Stretch Goals

- Combining multiple sensors
- Using RMS, FFT, or other signal processing
- Using machine learning for anomaly detection
- Showing motor-health trends
- Sending alerts through Wi-Fi or a cloud dashboard
- Monitoring multiple motors or sensor nodes

These are optional. Focus on getting the core detection working first.

Expected Demonstration

- Live motor monitoring
- Baseline for normal operation
- Physically introduced fault
- Detection of the abnormal condition
- Clear health status or alert
- Real-time response

Pre-recorded data alone should not be the main demonstration.

Expected Deliverables

Hardware

- Working motor monitoring setup
- Sensors and interface circuit
- Working prototype

Software

- Data acquisition
- Fault/anomaly detection
- Health status display or alert

Documentation

- Basic system architecture
- Sensors and detection method used
- Explanation of the fault condition tested

Evaluation Criteria

Category	What We Will Look At
Fault Detection	Can the system detect a real introduced fault?
Reliability	Does it consistently identify abnormal behaviour?
Response Time	How quickly does it detect the fault?
Detection Method	Quality of sensing and analysis
Health Status	Is the result easy to understand?
Working Prototype	Does the complete system work live?
Technical Depth	Quality of the engineering approach

What a Good Solution Looks Like

Build a low-cost system that monitors a running motor, learns its normal behaviour, detects a real change or fault, and alerts the user before it becomes a major failure.

You do not need to build a complete industrial predictive-maintenance product. Focus on making fault detection reliable and meaningful.

BEST OF LUCK!

PROBLEM STATEMENT 5

FROM STATIC DASHBOARDS TO AN AI-POWERED DATA ANALYST

24-Hour Hackathon

Background

Organisations generate large amounts of data through Excel files, CSVs, ERP systems, CRM systems, production databases, inventory systems, and financial software.

The data may already exist, but answering a new business question often still requires someone who knows Excel, SQL, or data-analysis tools.

Traditional dashboards show predefined charts and KPIs. But real users often ask questions that were not included when the dashboard was built.

The goal is to let users ask questions directly to their data and get an analysis without manually building a new report.

The Challenge

Build an AI-powered data analyst that can understand structured data, answer questions in natural language, perform the required analysis, and present the result as a useful chart, table, KPI, or explanation. The system should work with real-world data that may be spread across multiple sheets or tables.

The core idea is: Upload data → Understand it → Ask a question → Analyse the actual data → Visualise and explain the result.

What You Need to Build

1 • Data Ingestion

Allow users to upload Excel and CSV files. Support multiple sheets/tables in an Excel workbook. Teams may also support a structured database connection.

2 • Automatic Data Understanding

After upload, the system should identify important information such as tables, columns, data types, dates, numeric measures, categories, missing values, duplicates, and possible relationships between tables. For example, it may identify that Sales.Customer_ID matches Customers.Customer_ID and use that relationship during analysis.

3 • Natural-Language Questions

Users should not need to write SQL or configure pivot tables. They should be able to ask questions such as “Show me monthly revenue for 2026.” The system should understand the required table, measure, aggregation, filters, and time dimension, then run the calculation on the actual dataset.

4 • Dynamic Visualisation

The system should choose an appropriate way to present the result.

User Question	Possible Output
Revenue by month	Line chart
Sales by region	Bar chart
Top 10 products	Horizontal bar chart
Total revenue	KPI
Inventory trend	Line chart
Detailed records	Table

5 • AI-Generated Dashboard

A user should be able to ask something like “Analyse this dataset and create a management dashboard.” The system should identify useful KPIs, trends, comparisons, rankings, charts, tables, and possible alerts from the uploaded data.

6 • Conversational Follow-Up

The system should remember the current analysis so users can continue asking questions. For example: “Show revenue by region.” → “Only the last six months.” → “Compare it with the previous six months.” → “Which region contributed most to the decline?”

7 • Why? / Drill-Down Analysis

The system should ideally handle questions such as “Why did revenue decrease?” by looking at relevant dimensions such as region, product category, customer type, or time period and showing which factors contributed most.

8 • Data Quality Insights

The platform should identify common data issues such as missing values, duplicate IDs, negative or unusual values, and inconsistent date formats, and explain how they may affect the analysis.

Example

A company uploads a workbook containing Sales, Customers, Products, Inventory, and Returns. The user asks: “Create a dashboard showing our current business performance.” The system generates useful KPIs and charts from the actual data.

The user then asks: “Why did revenue fall last month?” followed by “Was it mainly because of a particular product category?” and “Show me the five products responsible for the largest decline.” Each question should lead to a new analysis without the user manually creating another report.

Critical Technical Challenge

The complete dataset should not simply be sent to an LLM. The system should use the AI to understand the question and decide what needs to be analysed, while a reliable data engine performs the actual calculations.

Teams may explore approaches such as schema-aware prompting, metadata retrieval, semantic layers, SQL generation, DataFrame operations, query planning, structured outputs, and conversational context.

Expected Demonstration

- Upload a multi-sheet or multi-table dataset
- Automatic schema and data-type detection
- Basic relationship and data-quality detection
- Ask questions in natural language
- Generate an analysis/query plan
- Execute the analysis on the actual dataset
- Automatically generate a suitable visualisation
- Generate a useful dashboard
- Ask follow-up questions while keeping context
- Perform a drill-down or “Why?” analysis
- Explain the results in human-readable language

Expected Deliverables

Platform

- Working web or desktop interface
- Data upload and understanding
- Natural-language analysis
- Dynamic visualisation
- Conversational follow-up

Data Engine / AI

- Schema and metadata handling
- Query/analysis generation
- Execution against actual data
- Result explanation

Documentation

- System architecture
- Supported data formats
- Brief explanation of the AI-to-data workflow

Evaluation Criteria

Category	What We Will Look At
Data Understanding	How well the system understands real datasets
Natural-Language Analysis	Accuracy in interpreting user questions
Correctness	Whether calculations use the actual data correctly
Visualisation	Whether the chosen charts communicate the result well
Follow-Up Analysis	Ability to maintain context across questions
Drill-Down	Ability to investigate and explain changes
Data Quality	Ability to identify important data issues
User Experience	Ease of asking questions and understanding results
Technical Depth	Quality of the AI-to-data architecture

What a Good Solution Looks Like

The core task is:

Build an AI-powered data analyst that can understand structured data, determine what the user is asking, perform the correct computation, choose a useful visualisation, explain the result, and continue the analysis conversationally.

The goal is not simply to build a chatbot that talks about an Excel file. The analysis must be performed on the actual data.

BEST OF LUCK!

PROBLEM STATEMENT 6

JET ENGINE ASSEMBLY CAD DESIGN & ASSEMBLY CHALLENGE

24-Hour Hackathon

Participants will be provided with reference views of the propeller, shaft and housing. Dimensions are intentionally not specified; participants must interpret the views and select their own dimensions while maintaining the geometry, proportions, symmetry and assembly relationships.

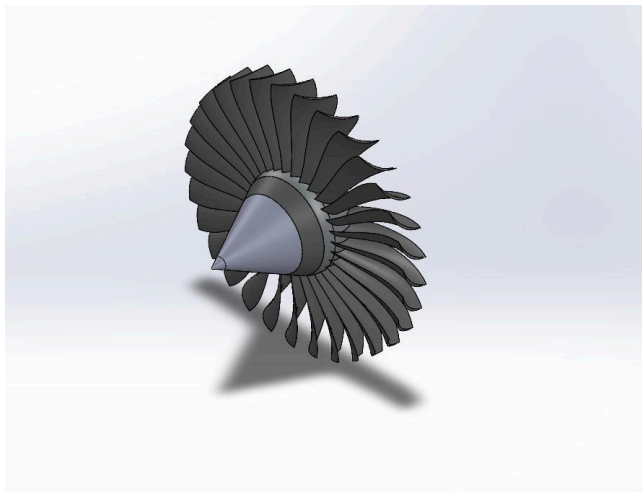
OBJECTIVE	EVALUATION
Interpret the supplied views and create accurate 3D geometry.	Understanding and effective use of CAD tools.
Determine suitable dimensions from the reference geometry.	Correct use of symmetry and pattern-based modelling.
Model the propeller, shaft and housing as individual components.	Quality and logical construction of the assembly.
Assemble the components into a coherent jet-engine model.	Clarity, creativity and professionalism of presentation.

Propeller

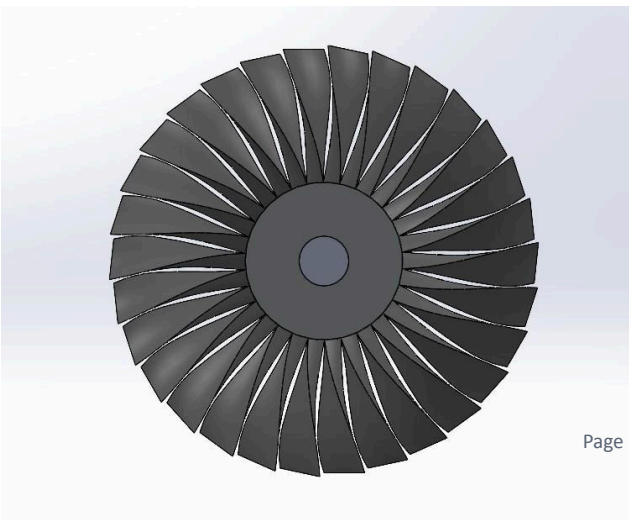
Reference View 1



Reference View 2

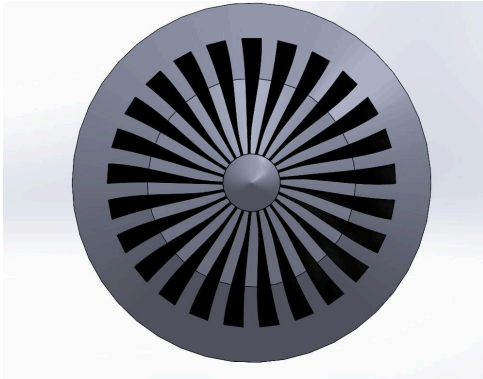


Reference View 3

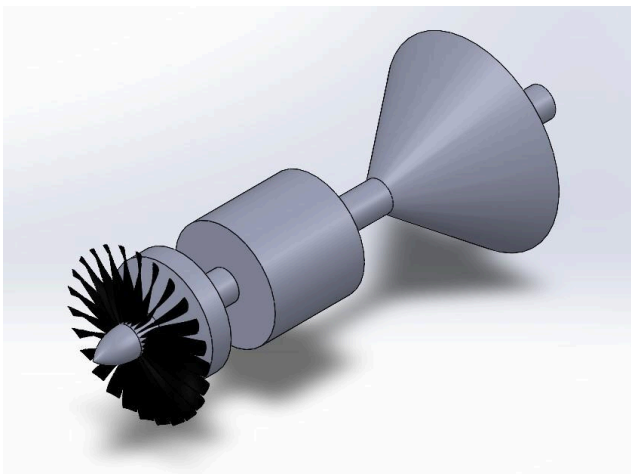


Shaft and Core

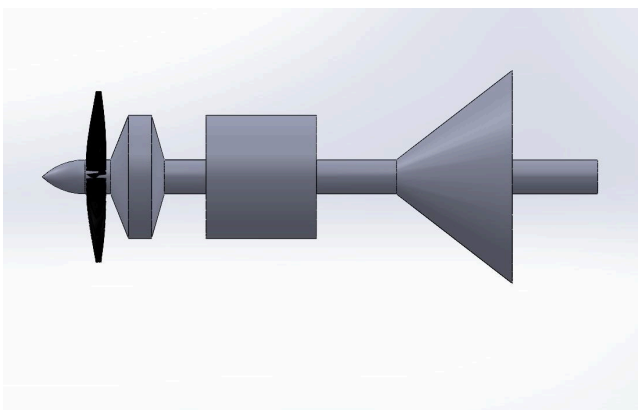
Reference View 1



Reference View 2

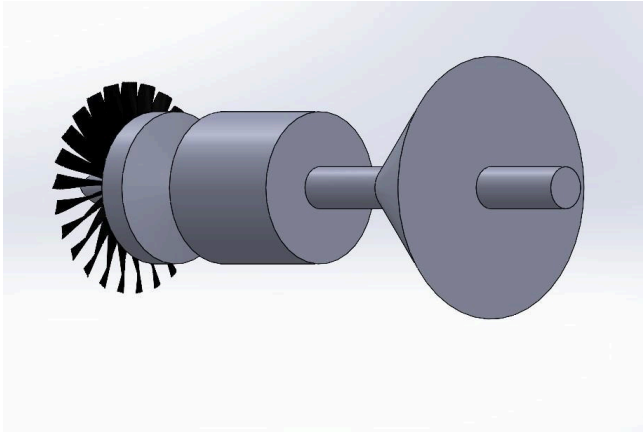


Reference View 3



Housing

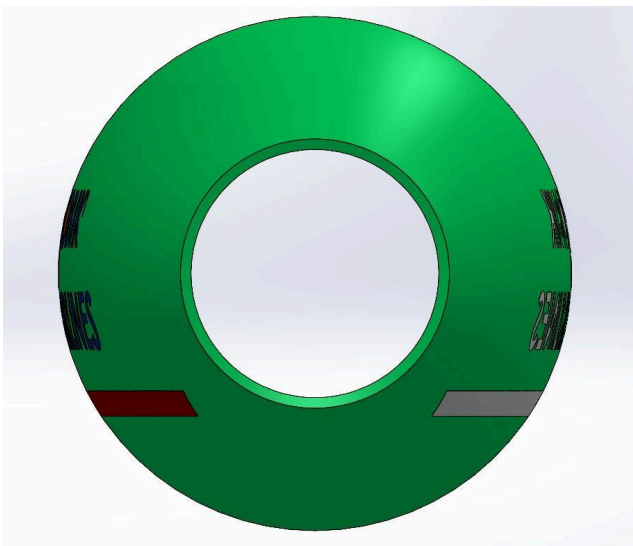
Reference View 1



Reference View 2

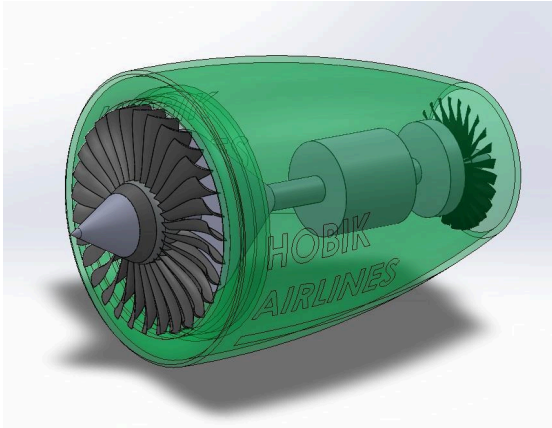


Reference View 3



Final Assembly

Reference View 1



Reference View 2



Reference View 3



Participant Instructions

- Use the supplied views as the primary reference for modelling.
- Dimensions may be selected by the participant; maintain consistent scale and proportions.
- Model the propeller, shaft and housing as separate components before assembling them.
- Demonstrate appropriate use of sketches, extrusions/revolves, patterns, fillets/chamfers and other CAD features as applicable.
- Pay particular attention to radial and rotational symmetry in the propeller and housing.
- Ensure the components are aligned and assembled logically.
- Present the final assembly using clear isometric and relevant orthographic views.

Winner Evaluation

The winner will be determined from the overall quality of the design and presentation, with emphasis on understanding of CAD tools, understanding and application of symmetry, interpretation of the reference views, assembly quality, design quality and presentation. The challenge is not intended to reward a particular set of dimensions; rather, it evaluates the participant's modelling decisions and ability to communicate the final design.

BEST OF LUCK!